

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Пермский национальный исследовательский политехнический
университет»

Электротехнический факультет

Кафедра: Информационные технологии и автоматизированные системы
(ИТАС)

Направление подготовки: 09.03.04 – «Программная инженерия»

ОТЧЕТ

Лабораторная работа №11.

Тема: «»

По дисциплине «Основы алгоритмизации и программирования»

Вариант 10

Выполнила: студентка группы РИС-25-26

Абрамова Валерия Андреевна

Принял: доц. Полякова О.А.

Пермь, 2026

Постановка задачи

Задача 1

1. Контейнер - вектор
2. Тип элементов - float

Задача 2

Тип элементов Money (см. лабораторную работу №3).

Задача 3

Параметризированный класс – Вектор (см. лабораторную работу №7)

Задача 4

Адаптер контейнера – очередь с приоритетами.

Задача 5

Параметризированный класс – Вектор

Адаптер контейнера - очередь с приоритетами.

Задание 3	Задание 4	Задание 5
Найти минимальный элемент и добавить его на заданную позицию контейнера	Найти элементы больше среднего арифметического и удалить их из контейнера	Каждый элемент домножить на максимальный элемент контейнера

Анализ задачи

Задача 1 (vector<float>)

1. Создаём вектор float для хранения вещественных чисел.
2. Вводим с клавиатуры 5 элементов и добавляем их в вектор.
3. Находим минимальный элемент перебором всех элементов вектора.
4. Запрашиваем у пользователя позицию для вставки минимального элемента.
5. Вставляем минимальный элемент в указанную позицию с помощью insert.
6. Выводим полученный вектор на экран.

Задача 2 (vector<Money>)

1. Создаём вектор Money для хранения денежных сумм.
2. Вводим с клавиатуры 3 суммы Money и добавляем их в вектор.
3. Вычисляем среднее арифметическое всех элементов вектора (переводим каждую сумму в double, суммируем, делим на количество).
4. Проходим по вектору с конца и удаляем все элементы, значение которых больше среднего арифметического.
5. Выводим оставшиеся элементы вектора.

Задача 3 (MyVector<Money>)

1. Используем шаблонный класс MyVector, который внутри хранит vector<T> и предоставляет методы add, print, removeGreaterThanAverage.
2. Создаём MyVector<Money> и вводим 3 суммы Money через метод add.
3. Вызываем метод removeGreaterThanAverage, который вычисляет среднее арифметическое (findAverage), затем удаляет элементы больше среднего.
4. Выводим результат через метод print.

Задача 4 (priority_queue<Money>)

1. Создаём priority_queue с компаратором greater<Money> (очередь с приоритетом по возрастанию).
2. Вводим 3 суммы Money и добавляем их в очередь через push.
3. Находим максимальный элемент, перебирая временную копию очереди.
4. Создаём новую очередь, проходим по исходной очереди, каждый элемент: если он не равен максимальному, умножаем его на максимальный (переводим в копейки, умножаем, переводим обратно в рубли и копейки); если равен максимальному, оставляем без изменений.
5. Выводим полученную очередь.

Задача 5 (MyPriorityQueue<Money>)

1. Используем шаблонный класс MyPriorityQueue, который внутри хранит priority_queue<T, vector<T>, greater<T>> и предоставляет методы add, print, multiplyByMax.
2. Создаём MyPriorityQueue<Money> и вводим 3 суммы Money через метод add.
3. Вызываем метод multiplyByMax: извлекаем все элементы в вектор, находим максимальный элемент, умножаем каждый элемент (кроме максимального) на максимальный, затем заново формируем очередь.
4. Выводим результат через метод print.

Код

```
#include <iostream>
#include <vector>
#include <queue>
#include <deque>
#include <locale>
#include <algorithm>
#include <functional>
```

```
using namespace std;
```

```
class Money {
private:
    long rubles;
    int kopeks;

public:
    Money() : rubles(0), kopeks(0) {}

    Money(long r, int k) : rubles(r), kopeks(k) {
        normalize();
    }
}
```

```
Money(const Money& other) : rubles(other.rubles), kopeks(other.kopeks) {}
```

```
~Money() {}
```

```
Money& operator=(const Money& other) {  
    if (this != &other) {  
        rubles = other.rubles;  
        kopeks = other.kopeks;  
    }  
    return *this;  
}
```

```
bool operator==(const Money& other) const {  
    return (rubles == other.rubles && kopeks == other.kopeks);  
}
```

```
bool operator!=(const Money& other) const {  
    return !(*this == other);  
}
```

```
bool operator>(const Money& other) const {  
    if (rubles != other.rubles) {  
        return rubles > other.rubles;  
    }  
    return kopeks > other.kopeks;  
}
```

```
bool operator<(const Money& other) const {  
    if (rubles != other.rubles) {  
        return rubles < other.rubles;  
    }  
    return kopeks < other.kopeks;  
}
```

```
bool operator>=(const Money& other) const {  
    return (*this > other) || (*this == other);  
}
```

```
bool operator<=(const Money& other) const {  
    return (*this < other) || (*this == other);  
}
```

```
Money operator+(const Money& other) const {  
    Money result;
```

```

    result.rubles = rubles + other.rubles;
    result.kopeks = kopeks + other.kopeks;
    result.normalize();
    return result;
}

```

```

Money operator*(int multiplier) const {
    Money result;
    long total = (rubles * 100 + kopeks) * multiplier;
    result.rubles = total / 100;
    result.kopeks = total % 100;
    if (result.kopeks < 0) {
        result.rubles--;
        result.kopeks += 100;
    }
    result.normalize();
    return result;
}

```

```

Money& operator+=(const Money& other) {
    rubles += other.rubles;
    kopeks += other.kopeks;
    normalize();
    return *this;
}

```

```

Money operator-(const Money& other) const {
    Money result;
    long total1 = rubles * 100 + kopeks;
    long total2 = other.rubles * 100 + other.kopeks;
    long diff = total1 - total2;
    result.rubles = diff / 100;
    result.kopeks = diff % 100;
    if (result.kopeks < 0) {
        result.rubles--;
        result.kopeks += 100;
    }
    return result;
}

```

```

Money operator--(int) {
    Money temp = *this;
    kopeks -= 1;
    normalize();
    return temp;
}

```

```
}
```

```
Money& operator--() {
```

```
    kopeks -= 1;
```

```
    normalize();
```

```
    return *this;
```

```
}
```

```
double toDouble() const {
```

```
    return rubles + kopeks / 100.0;
```

```
}
```

```
void normalize() {
```

```
    if (kopeks >= 100) {
```

```
        rubles += kopeks / 100;
```

```
        kopeks %= 100;
```

```
    }
```

```
    else if (kopeks < 0) {
```

```
        long newRubles = rubles + (kopeks / 100) - 1;
```

```
        int newKopeks = 100 + (kopeks % 100);
```

```
        if (newKopeks >= 100) {
```

```
            newRubles += newKopeks / 100;
```

```
            newKopeks %= 100;
```

```
        }
```

```
        rubles = newRubles;
```

```
        kopeks = newKopeks;
```

```
    }
```

```
}
```

```
friend istream& operator>>(istream& in, Money& m);
```

```
friend ostream& operator<<(ostream& out, const Money& m);
```

```
};
```

```
istream& operator>>(istream& in, Money& m) {
```

```
    cout << "Введите рубли: ";
```

```
    in >> m.rubles;
```

```
    cout << "Введите копейки: ";
```

```
    in >> m.kopeks;
```

```
    m.normalize();
```

```
    return in;
```

```
}
```

```
ostream& operator<<(ostream& out, const Money& m) {
```

```
    out << m.rubles << ",";
```

```
    if (m.kopeks < 10) {
```

```

        out << "0";
    }
    out << m.kopeks;
    return out;
}

```

```

template <typename T>
class MyVector {
private:
    vector<T> data;

public:
    MyVector() {}

    void add(const T& value) {
        data.push_back(value);
    }

    void insert(int index, const T& value) {
        if (index >= 0 && index <= (int)data.size()) {
            data.insert(data.begin() + index, value);
        }
    }

    void remove(int index) {
        if (index >= 0 && index < (int)data.size()) {
            data.erase(data.begin() + index);
        }
    }

    T get(int index) const {
        return data[index];
    }

    int size() const {
        return data.size();
    }

    void print() const {
        for (size_t i = 0; i < data.size(); i++) {
            cout << i + 1 << ". " << data[i] << endl;
        }
    }

    vector<T>& getData() {

```

```

    return data;
}

const vector<T>& getData() const {
    return data;
}

T findMin() const {
    if (data.empty()) {
        throw "Контейнер пуст!";
    }
    T minVal = data[0];
    for (size_t i = 1; i < data.size(); i++) {
        if (data[i] < minVal) {
            minVal = data[i];
        }
    }
    return minVal;
}

double findAverage() const {
    if (data.empty()) return 0;
    T sum;
    for (size_t i = 0; i < data.size(); i++) {
        sum = sum + data[i];
    }
    return sum.toDouble() / data.size();
}

void removeGreaterThanAverage() {
    if (data.empty()) return;
    double avg = findAverage();
    for (int i = data.size() - 1; i >= 0; i--) {
        if (data[i].toDouble() > avg) {
            data.erase(data.begin() + i);
        }
    }
}

void multiplyByMax() {
    if (data.empty()) return;
    T maxVal = data[0];
    for (size_t i = 1; i < data.size(); i++) {
        if (data[i] > maxVal) {
            maxVal = data[i];
        }
    }
}

```



```

    }
}
for (size_t i = 0; i < data.size(); i++) {
    if (data[i] != maxVal) {
        long total = (data[i].toDouble() * maxVal.toDouble()) * 100;
        long rub = total / 100;
        int kop = total % 100;
        data[i] = Money(rub, kop);
    }
}
};

```

```

template <typename T>
class MyPriorityQueue {
private:
    priority_queue<T, vector<T>, greater<T>> pq;

public:
    void add(const T& value) {
        pq.push(value);
    }

    void remove() {
        if (!pq.empty()) {
            pq.pop();
        }
    }

    T top() const {
        return pq.top();
    }

    int size() const {
        return pq.size();
    }

    bool empty() const {
        return pq.empty();
    }

    void print() const {
        priority_queue<T, vector<T>, greater<T>> temp = pq;
        int i = 1;
        while (!temp.empty()) {

```

```

        cout << i++ << ". " << temp.top() << endl;
        temp.pop();
    }
}

vector<T> toVector() const {
    vector<T> result;
    priority_queue<T, vector<T>, greater<T>> temp = pq;
    while (!temp.empty()) {
        result.push_back(temp.top());
        temp.pop();
    }
    return result;
}

T findMax() const {
    priority_queue<T, vector<T>, greater<T>> temp = pq;
    T maxVal = temp.top();
    while (!temp.empty()) {
        if (temp.top() > maxVal) {
            maxVal = temp.top();
        }
        temp.pop();
    }
    return maxVal;
}

void multiplyByMax() {
    vector<T> temp = toVector();
    T maxVal = findMax();
    while (!pq.empty()) pq.pop();
    for (size_t i = 0; i < temp.size(); i++) {
        if (temp[i] != maxVal) {
            long total = (temp[i].toDouble() * maxVal.toDouble()) * 100;
            long rub = total / 100;
            int kop = total % 100;
            pq.push(Money(rub, kop));
        }
        else {
            pq.push(temp[i]);
        }
    }
}
};

```

```

void task1() {
    cout << "\nЗАДАЧА 1" << endl;
    cout << "Контейнер: вектор\nТип: float\n" << endl;

    vector<float> vec;

    cout << "Введите 5 элементов (float):" << endl;
    for (int i = 0; i < 5; i++) {
        float val;
        cout << "Элемент " << i + 1 << ": ";
        cin >> val;
        vec.push_back(val);
    }

    cout << "\nИсходный вектор: ";
    for (float v : vec) cout << v << " ";
    cout << endl;

    float minVal = vec[0];
    for (float v : vec) {
        if (v < minVal) minVal = v;
    }
    cout << "Минимальный элемент: " << minVal << endl;

    int pos;
    cout << "Введите позицию для вставки минимального элемента (0-"
        << vec.size() << "): ";
    cin >> pos;

    if (pos >= 0 && pos <= (int)vec.size()) {
        vec.insert(vec.begin() + pos, minVal);
    }

    cout << "Результат: ";
    for (float v : vec) cout << v << " ";
    cout << endl;
}

void task2() {
    cout << "\nЗАДАЧА 2" << endl;
    cout << "Контейнер: вектор\nТип: Money\n" << endl;

    vector<Money> vec;

    cout << "Введите 3 суммы Money:" << endl;

```

```

for (int i = 0; i < 3; i++) {
    Money m;
    cout << "\nСумма " << i + 1 << ":" << endl;
    cin >> m;
    vec.push_back(m);
}

cout << "\nИсходный вектор:" << endl;
for (size_t i = 0; i < vec.size(); i++) {
    cout << i + 1 << ". " << vec[i] << endl;
}

double sum = 0;
for (Money m : vec) {
    sum += m.toDouble();
}
double avg = sum / vec.size();
cout << "\nСреднее арифметическое: " << avg << endl;

for (int i = vec.size() - 1; i >= 0; i--) {
    if (vec[i].toDouble() > avg) {
        cout << "Удален элемент: " << vec[i] << endl;
        vec.erase(vec.begin() + i);
    }
}

cout << "\nРезультат после удаления:" << endl;
if (vec.empty()) {
    cout << "Вектор пуст!" << endl;
}
else {
    for (size_t i = 0; i < vec.size(); i++) {
        cout << i + 1 << ". " << vec[i] << endl;
    }
}

}

void task3() {
    cout << "\nЗАДАЧА 3" << endl;
    cout << "Параметризированный класс: Вектор\nТип: Money\n" << endl;

    MyVector<Money> vec;

    cout << "Введите 3 суммы Money:" << endl;
    for (int i = 0; i < 3; i++) {

```

```

    Money m;
    cout << "\nСумма " << i + 1 << ":" << endl;
    cin >> m;
    vec.add(m);
}

cout << "\nИсходный вектор:" << endl;
vec.print();

vec.removeGreaterThanAverage();

cout << "\nРезультат после удаления элементов больше среднего:" << endl;
if (vec.size() == 0) {
    cout << "Вектор пуст!" << endl;
}
else {
    vec.print();
}
}

void task4() {
    cout << "\nЗАДАЧА 4" << endl;
    cout << "Адаптер контейнера: очередь с приоритетами\nТип: Money\n" <<
endl;

    priority_queue<Money, vector<Money>, greater<Money>> pq;

    cout << "Введите 3 суммы Money:" << endl;
    for (int i = 0; i < 3; i++) {
        Money m;
        cout << "\nСумма " << i + 1 << ":" << endl;
        cin >> m;
        pq.push(m);
    }

    cout << "\nИсходная очередь с приоритетами (по возрастанию):" << endl;
    priority_queue<Money, vector<Money>, greater<Money>> temp = pq;
    int idx = 1;
    while (!temp.empty()) {
        cout << idx++ << ". " << temp.top() << endl;
        temp.pop();
    }

    Money maxVal = pq.top();
    priority_queue<Money, vector<Money>, greater<Money>> temp2 = pq;

```

```

while (!temp2.empty()) {
    if (temp2.top() > maxVal) {
        maxVal = temp2.top();
    }
    temp2.pop();
}
cout << "\nМаксимальный элемент: " << maxVal << endl;

priority_queue<Money, vector<Money>, greater<Money>> newPq;
temp = pq;
while (!temp.empty()) {
    Money current = temp.top();
    temp.pop();
    if (current != maxVal) {
        long total = (current.toDouble() * maxVal.toDouble()) * 100;
        long rub = total / 100;
        int kop = total % 100;
        newPq.push(Money(rub, kop));
    }
    else {
        newPq.push(current);
    }
}
pq = newPq;

cout << "\nРезультат после умножения каждого элемента на максимальный:"
<< endl;
temp = pq;
idx = 1;
while (!temp.empty()) {
    cout << idx++ << ". " << temp.top() << endl;
    temp.pop();
}
}

void task5() {
    cout << "\nЗАДАЧА 5" << endl;
    cout << "Параметризированный класс: Вектор\nАдаптер: очередь с
приоритетами\nТип: Money\n" << endl;

    MyPriorityQueue<Money> mpq;

    cout << "Введите 3 суммы Money:" << endl;
    for (int i = 0; i < 3; i++) {
        Money m;

```

```

        cout << "\nСумма " << i + 1 << ":" << endl;
        cin >> m;
        mpq.add(m);
    }

    cout << "\nИсходная очередь с приоритетами:" << endl;
    mpq.print();

    mpq.multiplyByMax();

    cout << "\nРезультат после умножения каждого элемента на максимальный:"
    << endl;
    mpq.print();
}

int main() {
    setlocale(LC_ALL, "Russian");

    int choice;

    do {
        cout << "\nГЛАВНОЕ МЕНЮ" << endl;
        cout << "1. Задача 1 (vector<float>)" << endl;
        cout << "2. Задача 2 (vector<Money>)" << endl;
        cout << "3. Задача 3 (MyVector<Money>)" << endl;
        cout << "4. Задача 4 (priority_queue<Money>)" << endl;
        cout << "5. Задача 5 (MyPriorityQueue<Money>)" << endl;
        cout << "0. Выход" << endl;
        cout << "Ваш выбор: ";
        cin >> choice;

        switch (choice) {
            case 1:
                task1();
                break;
            case 2:
                task2();
                break;
            case 3:
                task3();
                break;
            case 4:
                task4();
                break;
            case 5:

```

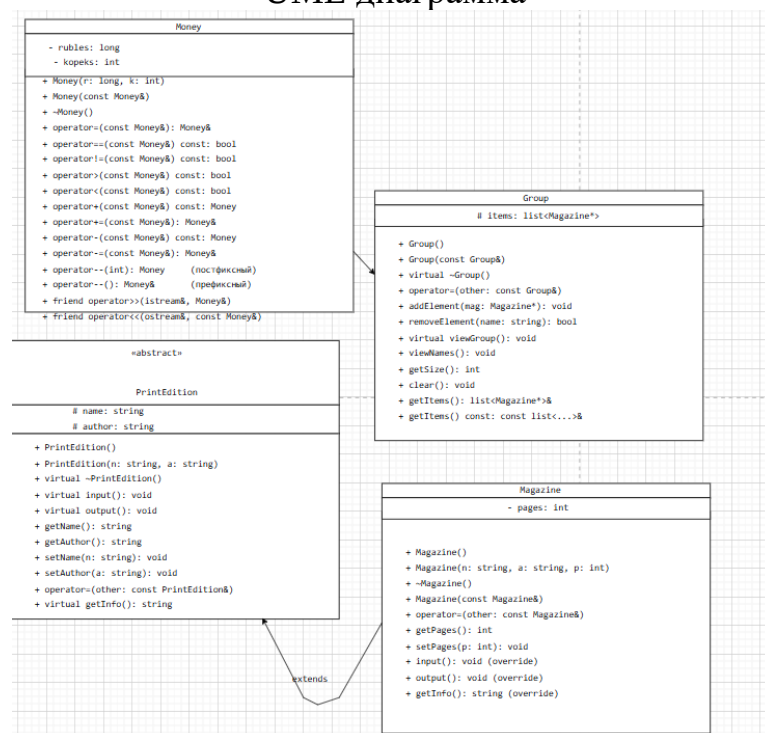
```

    task5();
    break;
case 0:
    cout << "Выход из программы..." << endl;
    break;
default:
    cout << "Неверный выбор!" << endl;
}
} while (choice != 0);

return 0;
}

```

UML диаграмма



Вывод программы


```
ГЛАВНОЕ МЕНЮ
1. Задача 1 (vector<float>)
2. Задача 2 (vector<Money>)
3. Задача 3 (MyVector<Money>)
4. Задача 4 (priority_queue<Money>)
5. Задача 5 (MyPriorityQueue<Money>)
0. Выход
Ваш выбор: 1

ЗАДАЧА 1
Контейнер: вектор
Тип: float

Введите 5 элементов (float):
Элемент 1: 5
Элемент 2: 10.5
Элемент 3: 20.3
Элемент 4: 5.2
Элемент 5: 30.1

Исходный вектор: 5 10.5 20.3 5.2 30.1
Минимальный элемент: 5
Введите позицию для вставки минимального элемента (0-5): 15.7
Результат: 5 10.5 20.3 5.2 30.1

ГЛАВНОЕ МЕНЮ
1. Задача 1 (vector<float>)
2. Задача 2 (vector<Money>)
3. Задача 3 (MyVector<Money>)
4. Задача 4 (priority_queue<Money>)
5. Задача 5 (MyPriorityQueue<Money>)
0. Выход
Ваш выбор: Выход из программы...
```

```
ГЛАВНОЕ МЕНЮ
1. Задача 1 (vector<float>)
2. Задача 2 (vector<Money>)
3. Задача 3 (MyVector<Money>)
4. Задача 4 (priority_queue<Money>)
5. Задача 5 (MyPriorityQueue<Money>)
0. Выход
Ваш выбор: 2

ЗАДАЧА 2
Контейнер: вектор
Тип: Money

Введите 3 суммы Money:

Сумма 1:
Введите рубли: 2
Введите копейки: 2

Сумма 2:
Введите рубли: 3
Введите копейки: 10

Сумма 3:
Введите рубли: 50
Введите копейки: 20

Исходный вектор:
1. 2,02
2. 3,10
3. 50,20

Среднее арифметическое: 18.44
Удален элемент: 50,20

Результат после удаления:
1. 2,02
2. 3,10
```

```
ЗАДАЧА 3
Параметризированный класс: Вектор
Тип: Money

Введите 3 суммы Money:

Сумма 1:
Введите рубли: 10
Введите копейки: 50

Сумма 2:
Введите рубли: 20
Введите копейки: 75

Сумма 3:
Введите рубли: 4
Введите копейки: 30

Исходный вектор:
1. 10,50
2. 20,75
3. 4,30

Результат после удаления элементов больше среднего:
1. 10,50
2. 4,30
```

```
Содержимое:
1. 100,50
2. 50,30

Введите значение для поиска: Введите рубли: 100
Введите копейки: 50
Увеличена запись 1: 102,00
Изменение завершено. Изменено записей: 1

МЕНЮ
1. Создать записи (ввод)
2. Просмотреть записи
3. Удалить записи больше заданного значения
4. Увеличить записи с заданным значением на 1 рубль 50 копеек
5. Добавить K записей после записи с номером N
6. Демонстрация операций -- (вычитание копеек)
0. Выход
Ваш выбор: 5

Содержимое:
1. 102,00
2. 50,30
```

Ответы на вопросы